

Atelier : Micro Service avec Docker

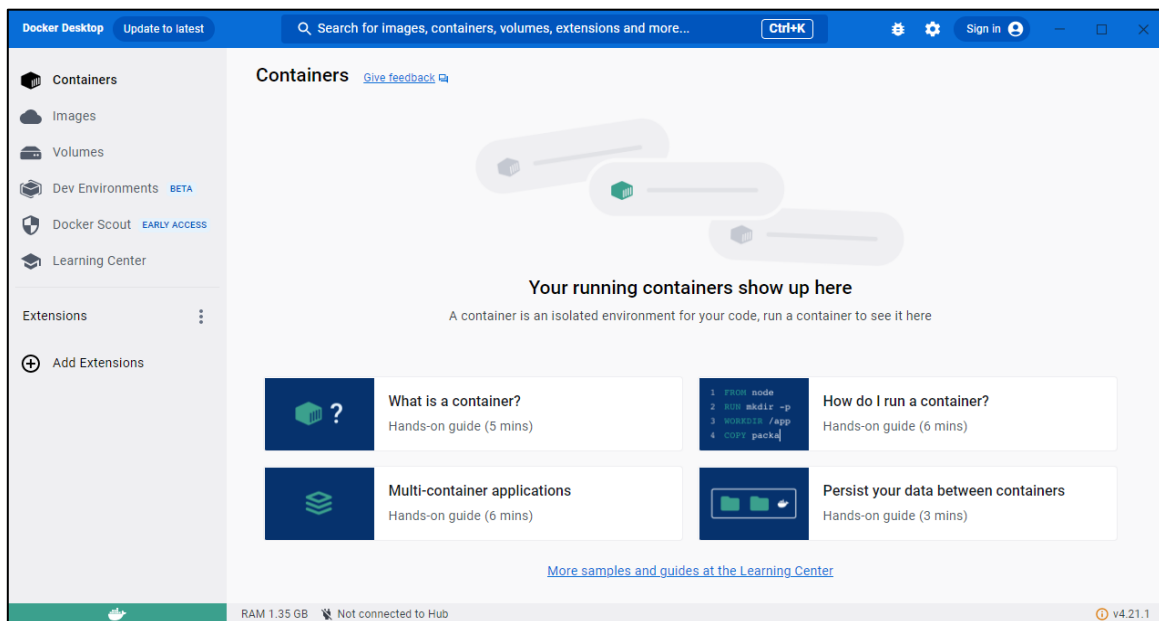
Objectifs

- Déployer un Micro service Spring Boot avec Docker.
- Découvrir comment pousser une image docker d'un Micro service sur le Hub.
- Utiliser l'outil de test Cloud « [KillerCoda](#) » pour tester l'image en ligne.

Partie I : Dockerisation d'une application MS sur Docker.

1. Installation Docker

Avant de déployer l'application sur Docker, assurez-vous d'avoir installé Docker. Dans cet exemple, nous avons utilisé Docker Desktop pour Windows.



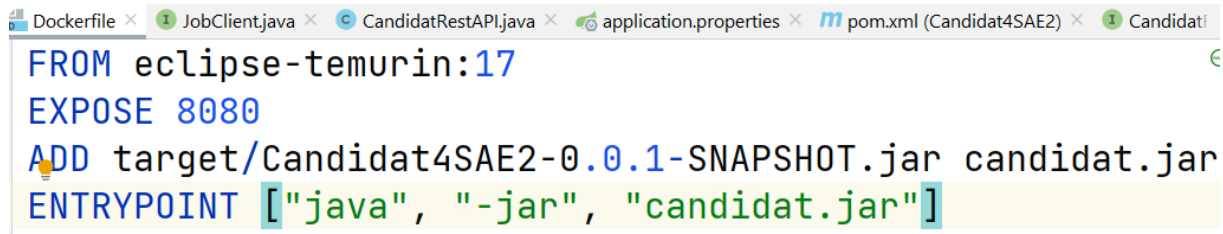
2. Déployer une application Spring Boot avec Docker

a. Création du fichier Dockerfile

- Une image Docker représente un élément important du travail avec le moteur Docker.
- Dans la structure de projet ci-dessous, nous avons créé un "Dockerfile". Docker lit les commandes / instructions de "Dockerfile" et construit l'image.

Nous commençons tout d'abord par la création d'un fichier « **dockerfile** » sous la racine du projet.

Ce fichier contient les commandes permettant de créer l'image docker à partir de notre application locale :



```
FROM eclipse-temurin:17
EXPOSE 8080
ADD target/Candidat4SAE2-0.0.1-SNAPSHOT.jar candidat.jar
ENTRYPOINT ["java", "-jar", "candidat.jar"]
```

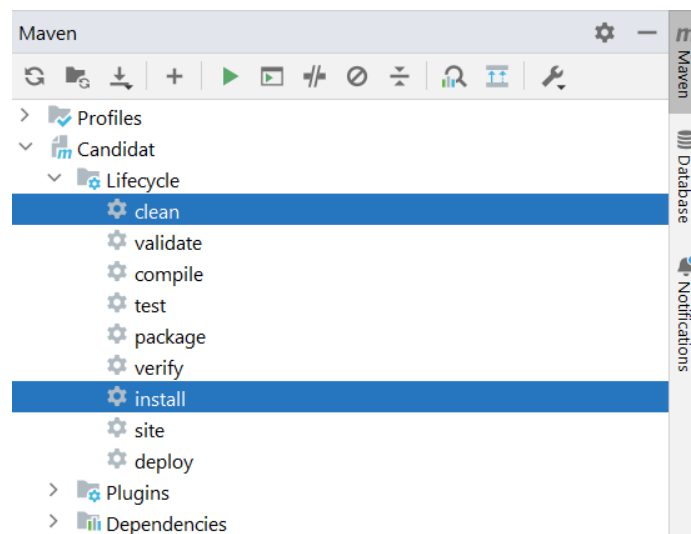
- **FROM** : Représente la première propriété dans le fichier Dockerfile. Cette commande va créer un calque à partir de l'image Docker. Dans notre cas, nous avons utilisé eclipse-temurin:17, ce qui signifie que cette application fonctionnera sous Java 17.
- **EXPOSE** : Exposer le port pour le noeud final . Dans cet exemple, nous avons utilisé 8080 en tant que port d'exécution de l'application locale.
- **ADD** : Cette commande permet de définir la source et la destination.
- **ENTRYPOINT** : C'est semblable à CMD, où le fichier de commande / jar sera exécuté.

b. Génération du fichier .jar avec mvn install

Maintenant, exécuter la commande pour construire l'image et déployez-la sur Docker.

Avant d'exécuter la commande Docker, nous devons créer le **fichier .jar** qui encapsule l'application entière (le code compilé de l'application, ainsi que les dépendances et les ressources nécessaires).

Nous devons donc utiliser les commandes maven « **clean puis install** » pour créer ce fichier **.jar** sous le dossier **target**.



c. Création de l'image Docker en utilisant ce fichier .jar

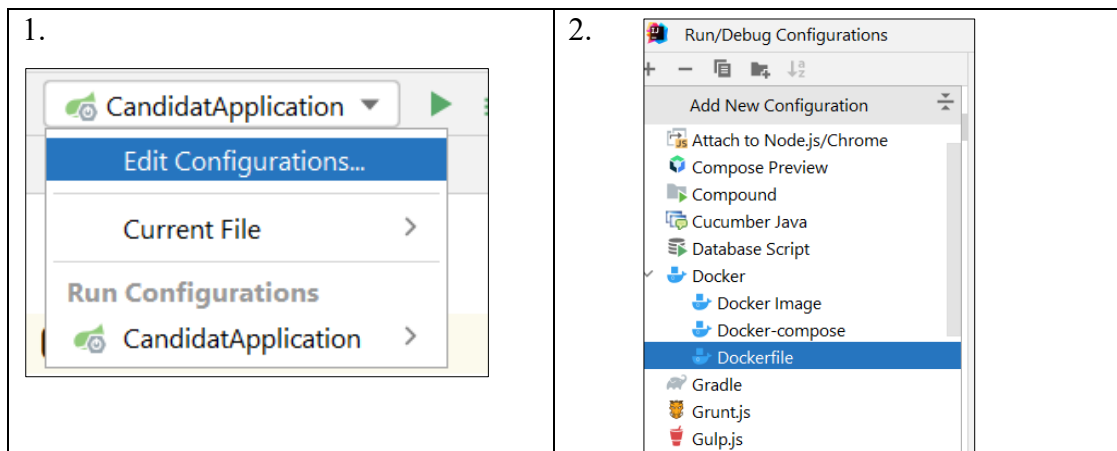
Ce fichier .jar généré va être maintenant copié dans l'image Docker et utilisé comme point d'entrée pour l'exécution de l'application lorsque le conteneur démarre.

Vous pouvez ainsi utiliser le terminal pour générer l'image. Dans ce cas lancer le cmd dans le répertoire qui contient le fichier Dockerfile, puis taper cette commande.

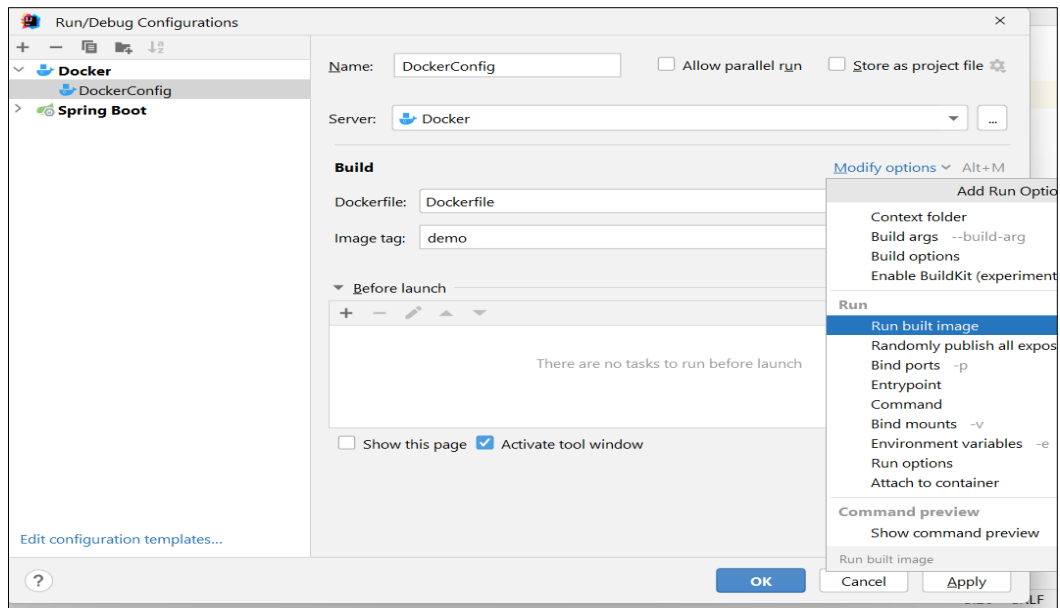
```
docker build -t demo .
```

Ou bien à travers la configuration suivante de l'IDE IntelliJ :

- i. Créer une nouvelle configuration de type Docker, en choisissant **Dockerfile** :



- ii. Nommer cette configuration, donner un nom à l'image docker à générer (**demo** dans notre cas), puis décocher l'option « **Run built image** » afin de ne pas exécuter automatiquement cette image et donc empêcher la génération d'une instance docker (container) :

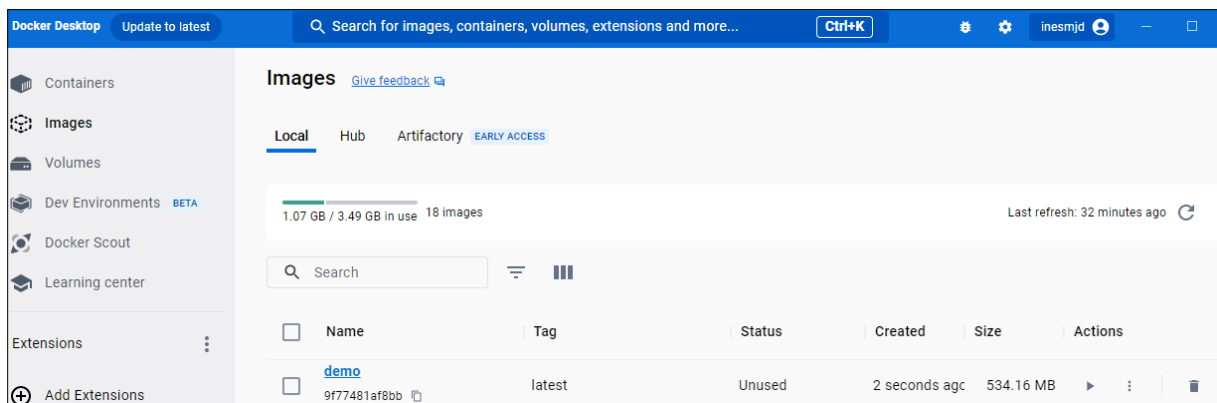


iii. Le résultat de l'exécution sera comme suit :

```
Build Log Dashboard
Step 1/4 : FROM openjdk:17
----> 5e28ba2b4cdb
Step 2/4 : EXPOSE 8080
----> Running in 545cca4409d1
Removing intermediate container 545cca4409d1
----> d88ebd184a89
Step 3/4 : ADD target/Candidat-0.0.1-SNAPSHOT.jar Candidat.jar
----> a8bca84a7d2e
Step 4/4 : ENTRYPOINT ["java", "-jar", "Candidat.jar"]
----> Running in 6670015fa272
Removing intermediate container 6670015fa272
----> 9f77481af8bb

Successfully built 9f77481af8bb
Successfully tagged demo:latest
'demo Dockerfile: Dockerfile' has been deployed successfully.
```

iv. Vérifier la création de l'image sur **Docker desktop** → **Images** → **Local** :



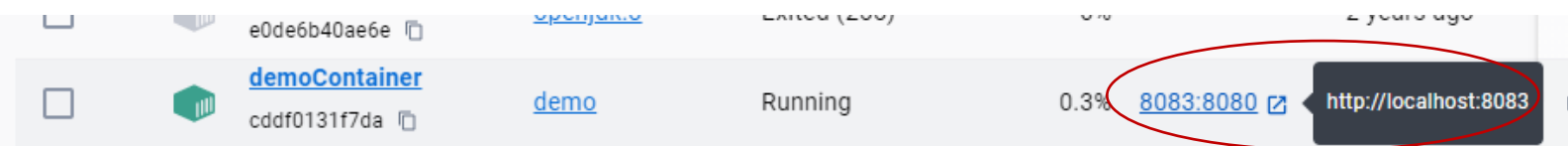
d. Dockerisation (conteneurisation) locale à travers Docker Desktop :

i. Vous pouvez utiliser le terminal pour générer l'image en se pointant sur le répertoire qui contient le fichier Dockerfile, puis taper cette commande.

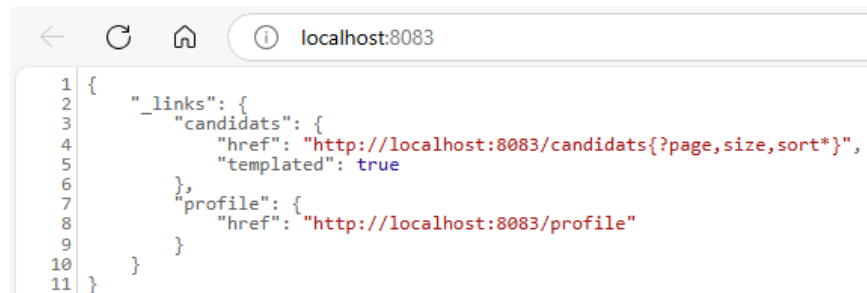
```
docker run -p 8083 :8080 --name demoContainer demo
```

- **8083** : le port de conteneurisation sur lequel l'application s'exécutera sur docker
- **8080** : le port d'exécution de l'application local
- **demoContainer** : le nom de l'instance exécutable (container) à générer et qui va être exécutée sur le port 8083
- **demo** : le nom de l'image docker créé dans l'étape précédente

La ligne suivante sera ajoutée sous **Docker desktop → Containers**



En cliquant maintenant sur le lien ci-dessus « **8083 :8080** » vous allez être redirigé vers la page web suivante :



Il suffit d'ajouter le path **/candidats** à la fin de cet URL afin d'afficher la liste des candidats :

```
localhost:8083/candidats
1 {
2   "_embedded": {
3     "candidats": [
4       {
5         "nom": "Mariem",
6         "prenom": "Ch",
7         "email": "ma@esprit.tn",
8         "_links": {
9           "self": {
10            "href": "http://localhost:8083/candidats/1"
11          },
12          "candidat": {
13            "href": "http://localhost:8083/candidats/1"
14          }
15        }
16      },
17      {
18        "nom": "Sarrah",
19        "prenom": "ab",
20        "email": "sa@esprit.tn",
21        "_links": {
22          "self": {
23            "href": "http://localhost:8083/candidats/2"
24          },
25          "candidat": {
26            "href": "http://localhost:8083/candidats/2"
27          }
28        }
29      },
30      {
31        "nom": "Mohamed",
32        "prenom": "ba",
33        "email": "mo@esprit.tn",
34        "_links": {
35          "self": {
36            "href": "http://localhost:8083/candidats/3"
37          },
38          "candidat": {
39            "href": "http://localhost:8083/candidats/3"
40          }
41        }
42      },
43      {
44        "nom": "Maroua",
45        "prenom": "dh",
46        "email": "maroua@esprit.tn",
47        "_links": {
48          "self": {
49            "href": "http://localhost:8083/candidats/4"
50          },
51          "candidat": {
52            "href": "http://localhost:8083/candidats/4"
53          }
54        }
55      }
56    ]
57  }
58 }
```

Tous les autres ressources REST sont maintenant accessibles à partir de cette instance Docker via le port **8083**.

ii. On peut utiliser aussi l'outil Docker desktop :

Docker desktop → Images → Local → cliquer sur l'action **run** devant l'image docker « demo » déjà créée afin de l'exécuter.

Fixer maintenant les paramètres de conteneurisation tel que le nom de l'instance à créer et le port d'exécution sur docker :



Run a new container
demo2:latest

Optional settings



Container name

demoContainer

A random name is generated if you do not provide one.

Ports

Enter "0" to assign randomly generated host ports.

Host port

8083

:8080/tcp

Volumes

Host path



Container path



Environment variables

Variable

Value



Cancel

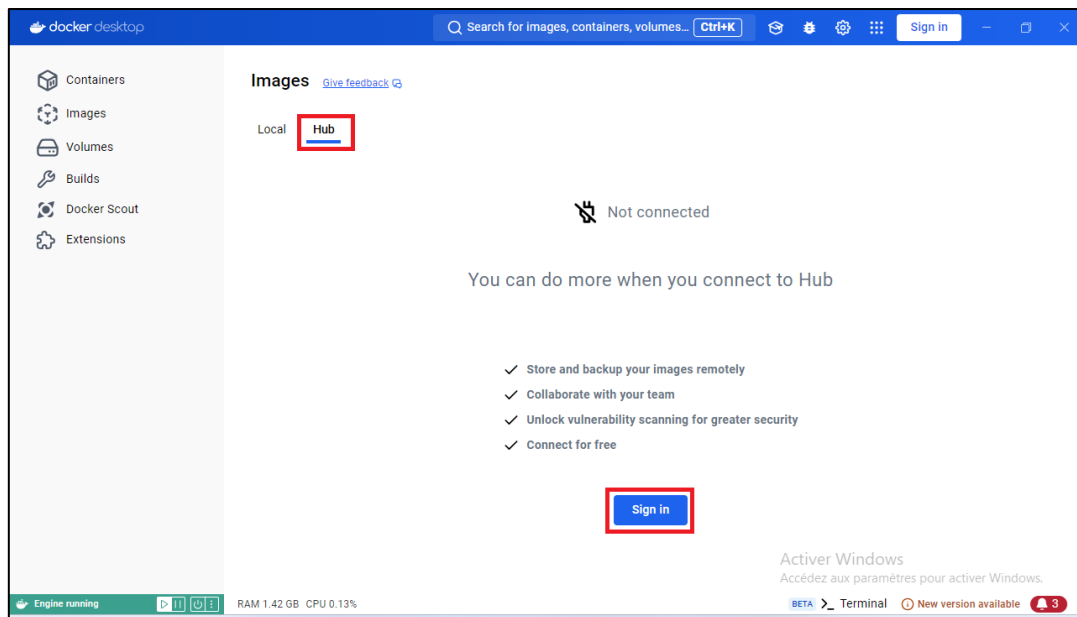
Run

Et l'instance exécutable (container) « **demoContainer** » sera générée et exécutée sous le port 8083.

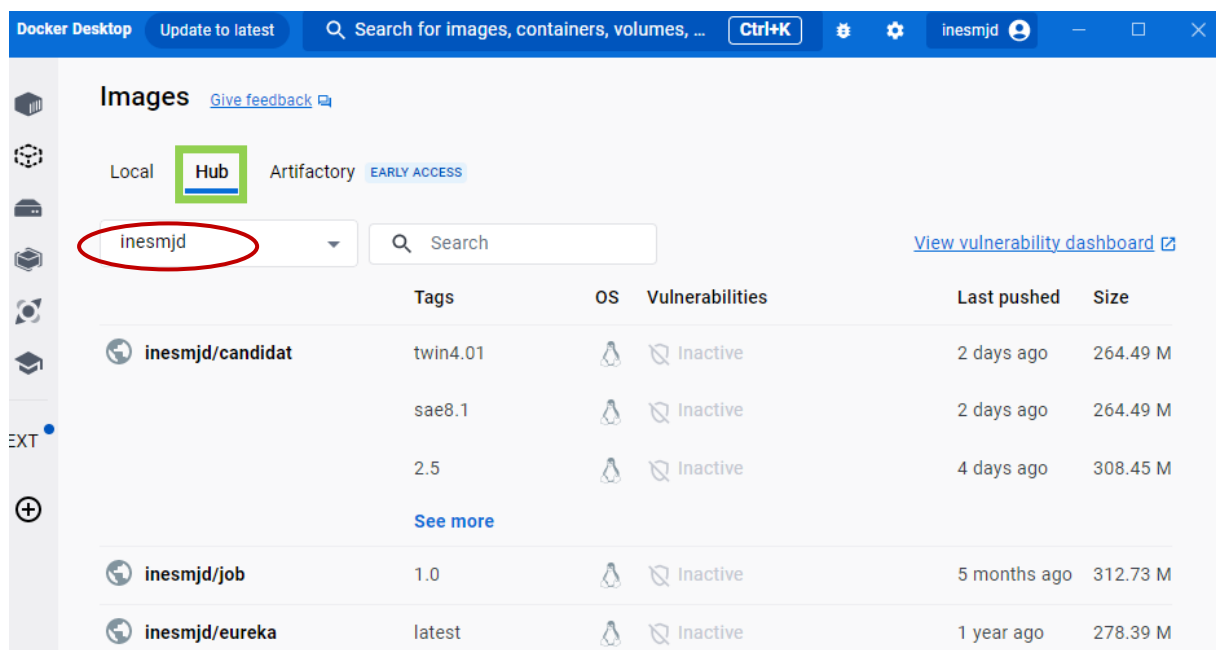
Partie II : Dockerisation avec docker Hub

Dans cette partie nous allons voir comment partager une image sur docker hub, et comment utiliser l'outil [KillerCoda](#) pour la tester en ligne.

Afin de se connecter au « **docker hub** », lancer docker desktop puis cliquer sur le lien « **hub** », par la suite cliquer sur « **sign in** », puis taper vos coordonnées (Username + Password).

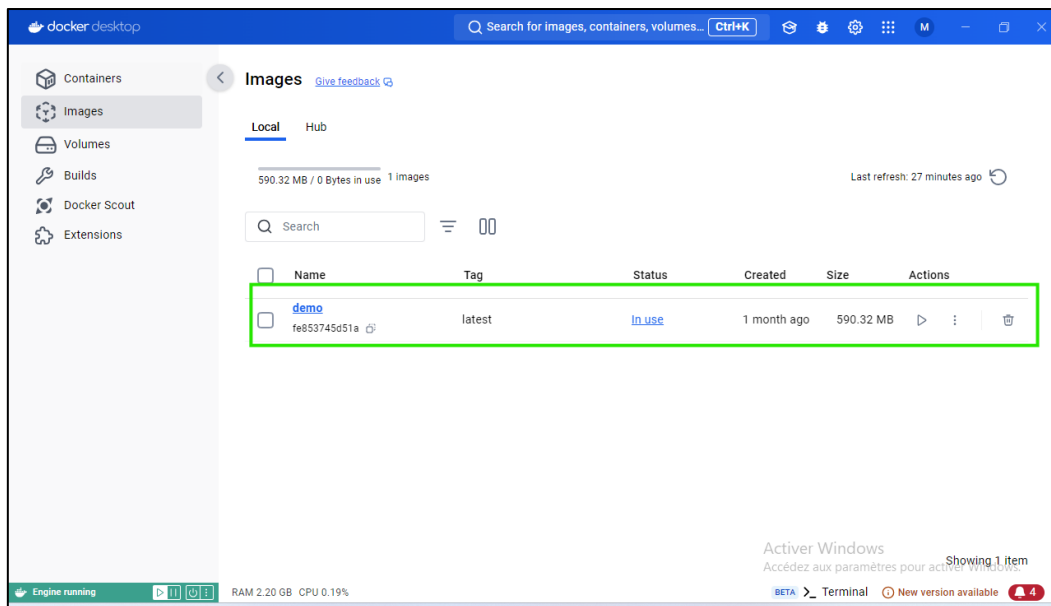


Si la connexion est établie, il faut avoir cette fenêtre :



Maintenant vous allez pousser (**push**) une image docker locale vers votre **Docker hub**.

L'image qu'on va utiliser pour cette étape est appelée « **demo** » (même image qu'on a utilisé dans la première partie en local).



Assurez-vous que vous êtes connecté sur **Docker hub**. Pour vérifier vous pouvez utiliser un terminal cmd en tapant la commande « **docker login** ».

```

C:\Users\Mohamed Amine\Downloads\CandidatProject (2)\CandidatProject>docker login
Authenticating with existing credentials...
Login Succeeded

C:\Users\Mohamed Amine\Downloads\CandidatProject (2)\CandidatProject>

```

Maintenant pour envoyer l'image sur le compte **docker hub** tapez la commande :

docker push demo

```

C:\Users\Mohamed Amine\Downloads\CandidatProject (2)\CandidatProject>docker login
Authenticating with existing credentials...
Login Succeeded

C:\Users\Mohamed Amine\Downloads\CandidatProject (2)\CandidatProject>docker push demo
Using default tag: latest
The push refers to repository [docker.io/library/demo]
88ea280d9ddd: Pushed
6b5aaff44254: Pushed
6b5aaff44254: Pushing [=====] 133MB/209.2MB
53a0b163e995: Pushed
9b55156abf26: Pushed
9b55156abf26: Pushing [=====] 130.5MB/152MB
293d5db30c9f: Pushed
9c742cd6c7a5: Pushed
1.0.0: digest: sha256:8a72576cd88c896ce3e219ce7d5a6ddb6f5abea2ed49561d8ab71e55cffe88bc size: 2007

```

En cas où vous recevrez un message d'erreur, c'est que vous devez « taguer » votre image avant de la pousser « push » sur docker hub :

i. Taguer l'image docker générée :

Le tagging est une étape importante pour pousser une image vers Docker Hub. Il permet d'identifier l'image et de spécifier où elle doit être stockée dans Docker Hub.

Lancer la commande suivante pour faire le push de votre image vers un repository sous votre espace docker Hub :

```
docker tag <image_name> <username>/<repository_name>:<tag>
```

où :

- **<image_name>** : le nom de l'image Docker
- **<username>** : Votre nom d'utilisateur Docker Hub
- **<repository_name>** : Le nom du dépôt sur Docker Hub, qui peut avoir généralement le même nom de l'image docker
- **<tag>** : Le tag que vous voulez donner à l'image (ex. latest ou v1.0.0, etc).

Dans notre cas, la commande à passer est la suivante :

```
docker tag demo inesmjd/demo:v1.0
```

ii. Pousser l'image taguée vers le docker hub :

Une fois l'image correctement taguée, vous pouvez la pousser vers Docker Hub avec la commande suivante :

```
docker push <username>/<repository_name>:<tag>
```

Dans notre cas, on lancera la commande suivante :

```
docker push inesmjd/demo:v1.0
```

Docker va commencer à envoyer (**push**) votre image vers Docker Hub.

NB : Cela peut prendre un certain temps en fonction de la taille de l'image et de votre connexion Internet.

L'image est maintenant sur « **docker hub** » :

Images [Give feedback](#)

Local **Hub** Artifactory [EARLY ACCESS](#)

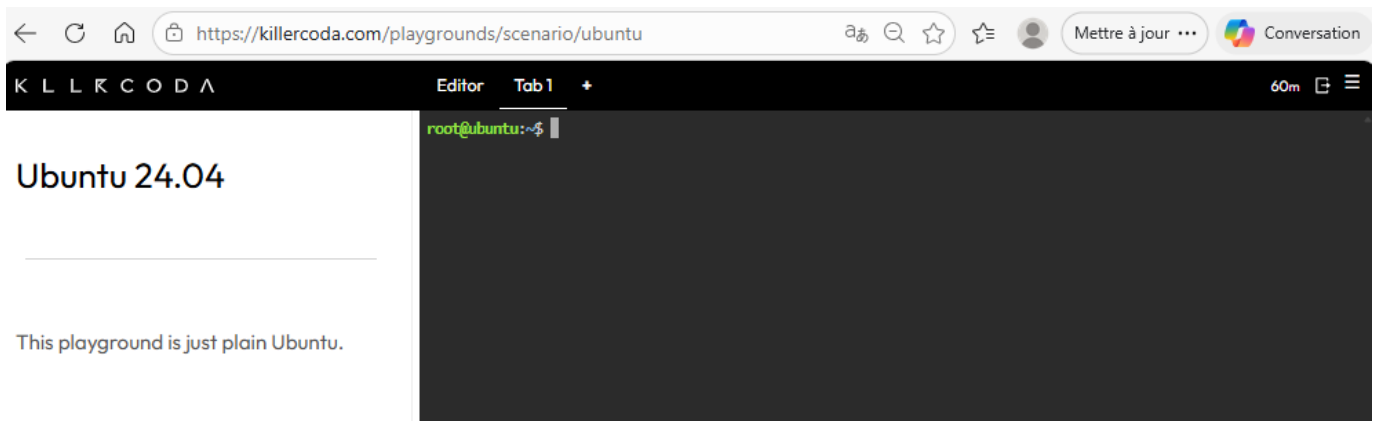
inesmjd [View vulnerability dashboard](#)

	Tags	OS	Vulnerabilities	Last pushed	Size
inesmjd/jobms	latest		Inactive	1 year ago	296.22 MB
inesmjd/demo2	latest		Inactive	2 years ago	299.26 MB
inesmjd/demo	latest		Inactive	2 years ago	299.26 MB
	v1.0		Inactive	2 years ago	299.26 MB

iii. Télécharger (pull) l'image docker Hub vers une plateforme cloud : KillerCoda

Pour tester l'image dans une machine distante, on va utiliser la plateforme cloud « **KillerCoda** ».

- Cliquer sur l'adresse [KillerCoda](#) pour accéder à la plateforme, puis accéder à **Playgrounds** → **Ubuntu**.
- On obtient un terminal Linux Docker prêt à l'emploi .



NB: Chaque instance de session dure 1h (60m comme affiché en haut à droite)

Dans KillerCoda, **on est dans une machine distante (cloud), qui n'a PAS accès aux images locales.**

Notre image `inesmjd/demo:v1.1` est dans le cloud (Docker Hub), elle peut être accessible depuis KillerCoda via la commande **`docker pull <username>/<repository_name>:<tag>`** comme suit :

```
K L L K C O D A Editor Tab1 +
Ubuntu 24.04
root@ubuntu:~$ docker pull inesmjd/demo:v1.0
v1.0: Pulling from inesmjd/demo
38a980f2cc8a: Pull complete
de849f1cfbe6: Pull complete
a7203ca35e75: Pull complete
adc9ef8bb4fa: Pull complete
Digest: sha256:a258280fc64ffa7f0f2e8bd04a3d07f978707fdb805396c671c0f84a7ca68987
Status: Downloaded newer image for inesmjd/demo:v1.0
docker.io/inesmjd/demo:v1.0
```

⇒ L'image est maintenant **récupérée depuis Docker Hub vers KillerCoda**

NB : Pour vérifier que l'image a été téléchargée sur l'environnement du test « KillerCoda » tapez : **docker image ls** ou **docker images**

iv. Démarrer l'image téléchargée : démarrer un container cloud

On va lancer l'exécution de l'image récupérée via la commande **docker run -d -p 8082:8080 --name candidat1 inesmjd/demo:v1.0**

```
root@ubuntu:~$ docker run -d -p 8082:8080 --name candidat1 inesmjd/demo:v1.0
fcb8607d5bc78ff29696e2a3ce25b059585d13904c5663a7e628674c079cc16d
root@ubuntu:~$ docker ps
CONTAINER ID   IMAGE             COMMAND                  CREATED        STATUS        PORTS                               NAMES
fcb8607d5bc7   inesmjd/demo:v1.0 "java -jar Candidat..." 10 seconds ago Up 9 seconds   0.0.0.0:8082->8080/tcp, [::]:8082->8080/tcp   candidat1
root@ubuntu:~$
```

NB :

- **votreRepository** : est le nom du repository dans docker hub.
- **image** : est le nom de l'image sur docker hub.
- **version** : est la version de l'image dans docker hub.
- Le container généré « **candidat1** » n'est pas accessible en localhost sur notre machine, mais il est accessible dans le réseau de la machine cloud (KillerCoda).

NB : Pour vérifier que le container a été démarré sur l'environnement du test « KillerCoda » tapez : **docker ps** ou **docker ps -a**

v. Tester le micro-service à partir de l'instance docker-hub démarrée

En fin pour tester l'exécution du micro-service candidat à partir de l'instance docker-hub :

❖ Afficher le résultat en terminal de KillerCoda :

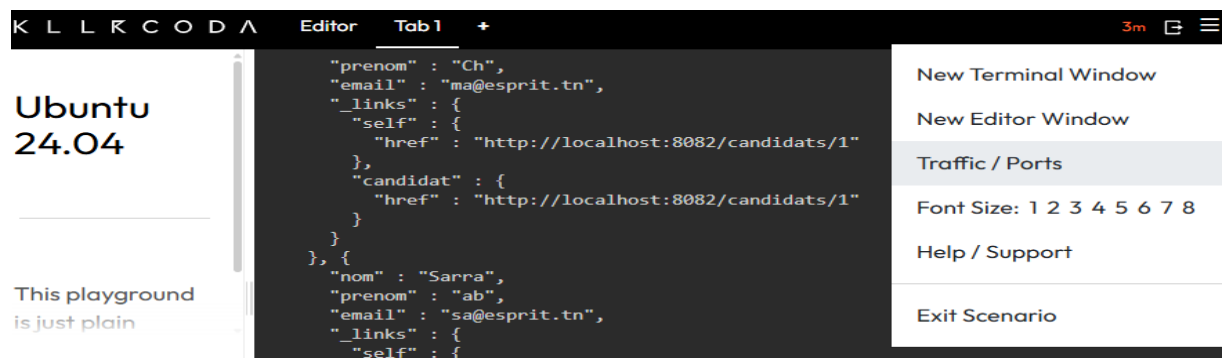
Utiliser la commande `curl http://localhost:8082/candidats` pour afficher la liste des candidats dans le terminal de killercoda en format JSON :

```
root@ubuntu:~$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
fcb8607d5bc7   inesmjd/demo:v1.0  "java -jar Candidat...."  10 seconds ago  Up 9 seconds  0.0.0.0:8082->8080/tcp
root@ubuntu:~$ ^C
root@ubuntu:~$ curl http://localhost:8082/candidats
{
  "_embedded" : {
    "candidats" : [ {
      "nom" : "Mariem",
      "prenom" : "Ch",
      "email" : "ma@esprit.tn",
      "_links" : {
        "self" : {
          "href" : "http://localhost:8082/candidats/1"
        },
        "candidat" : {
          "href" : "http://localhost:8082/candidats/1"
        }
      }
    }, {
      "nom" : "Sarrah",
      "prenom" : "ab",
      "email" : "sa@esprit.tn",
      "_links" : {
        "self" : {
          "href" : "http://localhost:8082/candidats/2"
        },
        "candidat" : {
          "href" : "http://localhost:8082/candidats/2"
        }
      }
    }, {
      "nom" : "Mohamed",
      "prenom" : "ba",
      "email" : "mo@esprit.tn",
      "_links" : {
        "self" : {
          "href" : "http://localhost:8082/candidats/3"
        },
        "candidat" : {
          "href" : "http://localhost:8082/candidats/3"
        }
      }
    }
  ]
}
```

NB : “localhost” fait référence ici à la machine cloud et non à notre ordinateur.

❖ Exposer les ports via l'interface (Traffic/Ports) :

- Cliquer sur le menu (Haut droit), puis choisir **Traffic / Ports** :



- Indiquer le port du container à tester via **Custom port**. Le port **8082** a été choisi comme port d'exposition de l'application container sur le cloud. Cliquer ensuite sur Access.

Traffic Port Accessor

Access HTTP services which run in your environment

The services need to run on all interfaces (like 0.0.0.0) and not just localhost

Host 1

Common Ports

80

8080

Custom Ports

8082

Access

- Le micro-service candidat sera affiché. Ajouter **/candidats** à l'adresse web pour voir la liste des candidats

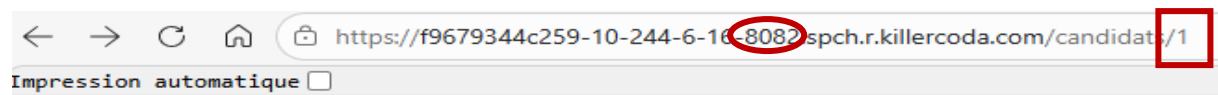
<http://ip172-18-0-49-crlsjtqim2rg0090j950-8082.direct.labs.play-with-docker.com/candidats>



```
{
  "_embedded": {
    "candidats": [
      {
        "nom": "Mariem",
        "prenom": "Ch",
        "email": "ma@esprit.tn",
        "_links": {
          "self": {
            "href": "http://f9679344c259-10-244-6-16-8082.spch.r.killercoda.com/candidats/1"
          },
          "candidat": {
            "href": "http://f9679344c259-10-244-6-16-8082.spch.r.killercoda.com/candidats/1"
          }
        }
      },
      {
        "nom": "Sarrah",
        "prenom": "ab",
        "email": "sa@esprit.tn",
        "_links": {
          "self": {
            "href": "http://f9679344c259-10-244-6-16-8082.spch.r.killercoda.com/candidats/2"
          },
          "candidat": {
            "href": "http://f9679344c259-10-244-6-16-8082.spch.r.killercoda.com/candidats/2"
          }
        }
      },
      {
        "nom": "Mohamed",
        "prenom": "ba",
        "email": "mo@esprit.tn",
        "_links": {

```

Pour chercher un candidat par id ajouter l'identifiant du candidat à la fin de l'URL :



```
{
  "nom" : "Mariem",
  "prenom" : "Ch",
  "email" : "ma@esprit.tn",
  "_links" : {
    "self" : {
      "href" : "http://f9679344c259-10-244-6-16-8082.spch.r.killercoda.com/candidats/1"
    },
    "candidat" : {
      "href" : "http://f9679344c259-10-244-6-16-8082.spch.r.killercoda.com/candidats/1"
    }
  }
}
```